

Rekurrente Graph Neural Networks

Logische Charakterisierungen mit Reellen Zahlen und Floats

Ahvonon, Heiman, Kuusisto, Lutz · NeurIPS 2024

Seminar „Graph Neural Networks“ · Universität Leipzig · Carsten Lutz

Persönliche Seminar-Notizen

Inhaltsverzeichnis

1. Die zentrale Frage	3
1.1. Warum wollen wir das wissen?	3
1.2. Die drei Erkenntnisse des Papers	3
2. GNN Grundlagen	4
2.1. Was ist ein GNN?	4
2.2. Normaler vs. Rekurrenter GNN	4
3. GNN[R] vs. GNN[F]	6
3.1. Der Unterschied auf einen Blick	6
3.2. Das Float-Problem: Warum Reihenfolge wichtig ist	6
3.3. Warum GNN[R] absolut stärker ist	7
4. Logische Charakterisierungen	8
4.1. Warum Logik?	8
4.2. Graded Modal Logic (GML)	8
4.3. Graded Modal Substitution Calculus (GMS)	8
4.4. ω -GML (für GNN[R])	9
4.5. Die zwei Haupttheoreme	9
4.6. Die Brücke: Distributed Automata (CMPAs)	9
5. MSO und das Überraschungsergebnis	11
5.1. Was ist MSO?	11
5.2. Das überraschende Resultat	11
5.3. Die Beweisidee (vereinfacht)	12
6. Das große Bild	13
6.1. Zusammenfassung aller Resultate	13
6.2. Praktische Implikationen	13
6.3. Offene Fragen	13
7. Vortrag Checkliste	14
7.1. Key Takeaways	14

Die zentrale Frage

Einstiegsfrage für den Vortrag

„Ihr trainiert einen GNN wochenlang und er lernt die Eigenschaft nicht. Liegt es am Training — oder ist es prinzipiell unmöglich für dieses Modell?“

Das ist keine rhetorische Frage. In der Praxis weiß man das oft nicht. Und genau das ist das Problem, das dieses Paper löst.

Was das Paper tut: Es charakterisiert exakt, was rekurrente GNNs ausdrücken können — und was nicht. Die Antwort kommt in Form von Logiken, weil Logik das präziseste Werkzeug ist um Ausdrucksstärke zu messen.

GNNs sind hier nicht das Ziel, sondern das Objekt der Untersuchung. Das Paper fragt nicht „wie trainiere ich einen guten GNN?“ sondern „was ist die fundamentale Ausdrucksstärke von rekurrenten GNNs?“.



Kernaussage

Wenn $\text{GNN} \equiv \text{Logik}$, dann weiß ich exakt was der GNN kann und was nicht. GNNs sind das **Was**, Logik ist das **Womit man es misst**.

Warum wollen wir das wissen?

- Debugging: Wenn ein GNN eine Eigenschaft nicht lernt — liegt es am Training oder ist es prinzipiell ausgeschlossen? Ohne Theorie weiß man das nicht.
- Modellwahl: Die Theorie sagt dir welches GNN-Modell du überhaupt wählen sollst. Brauchst du rekurrente Iteration? Reichen Floats?
- Effizienz: Wenn du weißt dass dein Modell eine Eigenschaft prinzipiell nicht ausdrücken kann, verschwendest du keine Zeit mit Training.

Die drei Erkenntnisse des Papers

Erwartet

$\text{GNN}[\mathbb{R}]$ ist mächtiger als $\text{GNN}[\mathbb{F}]$. Reelle Zahlen können mehr als Floats.

Interessant

Man kann beiden GNN-Typen exakt eine Logik zuordnen:
 $\text{GNN}[\mathbb{F}] \leftrightarrow \text{GMSC}$, $\text{GNN}[\mathbb{R}] \leftrightarrow \omega\text{-GML}$

Überraschend

Relativ zu MSO verschwinden alle Unterschiede zwischen $\text{GNN}[\mathbb{R}]$ und $\text{GNN}[\mathbb{F}]$ komplett.

GNN Grundlagen

Was ist ein GNN?

Ein GNN läuft auf einem Graphen. Jeder Knoten hat einen Feature-Vektor — eine Liste von Zahlen die seinen aktuellen Zustand beschreibt.

In jeder Runde passiert folgendes für jeden Knoten v :

1. AGGREGATE: Sammle die Feature-Vektoren aller Nachbarn von $v \rightarrow$ ergibt ein Multiset
2. COMBINE: Kombiniere deinen eigenen Vektor mit dem aggregierten Nachbar-Vektor $\rightarrow$ neuer Feature-Vektor

Formal:

$$x_v^t = \text{COM}(x_v^{t-1}, \text{AGG}(\{x_u^{t-1} \mid (v, u) \in E\}))$$

Das wird Runde für Runde wiederholt. Am Ende schaut man ob der Knoten in einem „accepting“ Zustand ist.

Normaler vs. Rekurrenter GNN

Normaler GNN

- Läuft genau **N Runden** (N fest vorgegeben)
- N ist ein Hyperparameter
- Nach N Runden: accepting oder nicht?
- Problem: N muss man im Voraus wissen

Rekurrenter GNN (RGNN)

- Kein festes N
- Läuft bis ein Knoten einen **accepting feature vector** erreicht
- Der GNN entscheidet selbst wann er fertig ist
- Terminierungsbedingung: $x_v^t \in F$ für irgendein t

► Formale Definition

Ein rekurrenter GNN[R] über (Π, d) ist ein Tupel $\mathcal{G} = (\mathbb{R}^d, \pi, \delta, F)$ wobei:

- $\pi : \mathcal{P}(\Pi) \rightarrow \mathbb{R}^d$ die Initialisierungsfunktion ist
- $\delta : \mathbb{R}^d \times \mathcal{M}(\mathbb{R}^d) \rightarrow \mathbb{R}^d$ die Übergangsfunktion
- $F \subseteq \mathbb{R}^d$ die Menge der accepting feature vectors

Der RGNN **akzeptiert** (G, v) wenn $x_v^t \in F$ für **irgendein** $t \in \mathbb{N}$.

► Konkretes Beispiel: Erreichbarkeit

Erreichbarkeit von Label p : „Gibt es einen Weg von v zu einem Knoten mit p ?“

R-simple RGNN über $(\Pi, 1)$ mit $C = A = 1$, $b = 0$:

- **Runde 0:** Zustand von w ist 1 wenn $p \in \lambda(w)$, sonst 0
- **Runde t :** Zustand ist 1 wenn w selbst 1 hat ODER ein Nachbar 1 hat
- Das propagiert durch den ganzen Graphen — egal wie groß er ist!

GMSC-Programm dafür: $X(\emptyset) :- p$ und $X :- \diamond X$

GNN[R] vs. GNN[F]

Der Unterschied auf einen Blick

Merkmal	GNN[R]	GNN[F]
Zahlen	Echte reelle Zahlen $\mathbb{R}$	Floating-Point (wie Computer)
Präzision	Beliebig genau	Begrenzt, gerundet
Aggregation	Unbeschränkt	Bounded (ab Bound k egal)
Ausdrucksstärke	Stärker	Schwächer
Praxis-relevanz	Theoretisch	Praktisch (echter Code)

Das Float-Problem: Warum Reihenfolge wichtig ist

Floating-Point Addition ist nicht assoziativ:

$$(a + b) + c \neq a + (b + c)$$

⚠ Achtung

Konkretes Beispiel aus dem Paper (Zahlensystem mit 2 Dezimalstellen, Basis 10):

$$1 + (-1) + 0.01 = 0 + 0.01 = 0.01$$

Aber:

$$(1 + 0.01) + (-1) = 1.0 + (-1) = 0$$

weil 1.01 in diesem System nicht darstellbar ist und gerundet wird!

Das bedeutet: Ein GNN[F] der Nachbarn summiert bekommt unterschiedliche Ergebnisse je nach Reihenfolge der Summation — das verletzt Isomorphie-Invarianz.

► Die Lösung: Aufsteigende Sortierung

Um Isomorphie-Invarianz zu retten sortiert man Floats vor der Summation (aufsteigend). Das ist in der Praxis üblich und genau definiert.

Aber: Dadurch entsteht Proposition 2.3:

Proposition 2.3

Für jedes Floating-Point System S gibt es ein $k \in \mathbb{N}$ sodass für alle Multisets M über Floats in S gilt:

$$\text{SUM}_S(M) = \text{SUM}_S(M|_k)$$

Das heißt: Ab k Kopien eines Wertes macht eine weitere Kopie keinen Unterschied mehr.

Konsequenz: $\text{GNN}[\text{F}]$ kann nicht beliebig genau zählen. Ab Bound k verliert er die Information.

Warum $\text{GNN}[\text{R}]$ absolut stärker ist

Ein $\text{GNN}[\text{R}]$ kann ausdrücken: „**Die Anzahl der direkten Nachbarn ist eine Primzahl.**“

Für jeden beliebigen Test $U \subseteq \mathbb{N}$ — egal wie kompliziert, sogar **unentscheidbar** — kann ein $\text{GNN}[\text{R}]$ prüfen ob die Nachbarzahl in U liegt.

Ein $\text{GNN}[\text{F}]$ mit Bound k kann Graphen mit k und $k + 1$ Nachbarn nicht unterscheiden.



Kernaussage

Absolut gilt: $\text{GNN}[\text{F}] < \text{GNN}[\text{R}]$

Die Extramacht liegt aber in einem Bereich den man in der Praxis nie braucht — das ist das überraschende Resultat am Ende.

Logische Charakterisierungen

Warum Logik?

Logik ist das präziseste Werkzeug für Ausdrucksstärke:

- Wenn GNN $\equiv$ Logik $\rightarrow$ ich weiß exakt was der GNN kann
- Ich kann Logik-Beweise nutzen um über GNNs nachzudenken
- Ich kann entscheiden ob eine Eigenschaft ausdrückbar ist oder nicht

Graded Modal Logic (GML)

GML ist eine Modallogik die **zählen** kann. Syntax:

$$\varphi ::= \top \mid p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \Diamond_{\geq k} \varphi$$

Die zentrale Formel $\Diamond_{\geq k} \varphi$ bedeutet:

„Mindestens k Nachbarn erfüllen φ “

Beispiele:

- $\Diamond_{\geq 3} p$ = „Dieser Knoten hat mindestens 3 Nachbarn mit Eigenschaft p “
- $\Diamond_{\geq 1} (p \wedge \Diamond_{\geq 2} q)$ = „Es gibt einen Nachbar mit p der selbst ≥ 2 Nachbarn mit q hat“

GML ist eine Erweiterung normaler Modallogik um dieses Zählen.

Graded Modal Substitution Calculus (GMSC)

GMSC erweitert GML um rekursive Regeln. Ein GMSC-Programm:

```
X(0) :-  $\varphi$            // Terminierungsklausel: Startzustand von X
X      :-  $\psi$           // Iterationsklausel: Update-Regel für X

wobei  $\varphi$  eine GML-Formel ist und  $\psi$  ein Schema das  $X$  enthalten kann (Rekursion!).
```

Die n -te Iterationsformel X^n von X ist definiert als:

- $X^0 = \varphi$ (Terminierungsklausel)
- $X^{n+1} = \psi$ wobei jedes X in ψ durch X^n ersetzt wird

GMSC akzeptiert (G, v) wenn $G, v \models X^n$ für **irgendein** $n \in \mathbb{N}$.

► Beispiele für GMSC-Programme

Erreichbarkeit von p :

$X(\emptyset) :- p \quad X :- \diamond X$

$X^i = \diamond \dots \diamond p$ (genau i Diamonds) = Erreichbarkeit in i Schritten

Zentrumspunkt-Eigenschaft (nicht in MSO ausdrückbar!):

$X(\emptyset) :- \perp \quad X :- \diamond X \wedge \Box X$

„Es gibt ein n sodass jeder gerichtete Pfad von w in genau n Schritten zu einem Knoten ohne Nachfolger führt.“

ω -GML (für $\text{GNN}[\mathbf{R}]$)

ω -GML ist einfacher zu erklären:

$$\varphi ::= \psi \mid \bigcup_{\psi \in \Psi} \psi$$

Es sind GML-Formeln plus unendliche Disjunktionen von GML-Formeln. Da $\text{GNN}[\mathbf{R}]$ mit reellen Zahlen beliebig viel unterscheiden kann, braucht man diese unendliche Ausdrucksstärke.

Proposition 2.6: GMSC ist in ω -GML ausdrückbar, aber nicht umgekehrt.

GMSC-Programme terminieren immer (zyklisches Wiederholen). ω -GML kann unentscheidbare Eigenschaften ausdrücken.

Die zwei Haupttheoreme

Theorem 3.2 — $\text{GNN}[\mathbf{F}] \equiv \text{GMSC}$

Die folgenden drei haben dieselbe Ausdrucksstärke:

- $\text{GNN}[\mathbf{F}]$ s
- GMSC
- R-simple aggregate-combine $\text{GNN}[\mathbf{F}]$ s

Absolut — ohne Einschränkung auf eine Hintergrundlogik.

Theorem 3.4 — $\text{GNN}[\mathbf{R}] \equiv \omega\text{-GML}$

$\text{GNN}[\mathbf{R}]$ s haben dieselbe Ausdrucksstärke wie ω -GML.

Absolut. Da ω -GML unentscheidbare Eigenschaften ausdrücken kann, ist $\text{GNN}[\mathbf{R}]$ sehr mächtig — aber jetzt wissen wir exakt wie mächtig.

Die Brücke: Distributed Automata (CMPAs)

Die Beweise gehen über eine Zwischenstufe: Counting Message Passing Automata (CMPAs). Das sind verteilte Automaten die wie GNNs funktionieren — jeder Knoten updatet seinen Zustand basierend auf dem Multiset der Nachbar-Zustände.

$$\begin{aligned} \text{GNN[F]} &\leftrightarrow \text{bounded FCMPA} \leftrightarrow \text{GMSC} \\ \text{GNN[R]} &\leftrightarrow \text{CMPA} \leftrightarrow \omega\text{-GML} \end{aligned}$$

MSO und das Überraschungsergebnis

Was ist MSO?

MSO = Monadic Second-Order Logic. Erweiterung von First-Order Logic:

- FO: Quantifizierung über einzelne Elemente ($\exists x, \forall x$)
- MSO: zusätzlich Quantifizierung über **Mengen** von Elementen ($\exists X, \forall X$)

► Was kann MSO ausdrücken?

MSO kann sehr viele natürliche Grapheigenschaften ausdrücken:

- Zusammenhängigkeit
- k -Färbbarkeit
- Existenz eines Hamiltonpfades
- Bipartitheit
- Planare Graphen (für festes Geschlecht)
- Viele weitere natürliche Eigenschaften

MSO ist in gewissem Sinne die „Grenze“ dessen was man sinnvoll über Graphen sagen will. Es ist die **natürliche** Logik für Graphen.

Das überraschende Resultat

Theorem 4.3 — Das Hauptresultat

Sei P eine Eigenschaft die in MSO ausdrückbar ist. Dann:

$$P \text{ ausdrückbar als GNN[R]} \Leftrightarrow P \text{ ausdrückbar als GNN[F]}$$

Das gilt auch für constant-iteration GNNs.

► Was bedeutet das intuitiv?

Absolut gesehen:

$$\text{GNN[R]} > \text{GNN[F]}$$

Die Extramacht liegt bei Eigenschaften wie „Nachbarzahl ist Primzahl“ oder sogar unentscheidbare Eigenschaften.

Relativ zu MSO:

$$\text{GNN[R]} = \text{GNN[F]}$$

Für alles was MSO beschreiben kann — also alle praktisch relevanten Eigenschaften — sind beide gleich stark.



Kernaussage

Die Extramacht von GNN[R] liegt komplett **außerhalb von MSO**. Da man in der Praxis nur MSO-definierbare Eigenschaften braucht: Floats reichen immer.

Die Beweisidee (vereinfacht)

Der Beweis nutzt **Parity Tree Automata (PTAs)** — Automaten die MSO auf Baumgraphen charakterisieren (Janin-Walukiewicz Theorem).

MSO-Eigenschaft P
↓ Theorem 4.6
PTA A
↓ Lemma 4.7
 k -Prefix Dekorationen von T
↓ Lemma C.4
GMSC-Programm Λ

Das zeigt: Wenn P in MSO und als $\text{GNN}[\text{R}]$ ausdrückbar ist, dann auch in GMSC, und damit auch als $\text{GNN}[\text{F}]$.

Das große Bild

Zusammenfassung aller Resultate

Absolut:

$$\text{GNN[F]} \equiv \text{GMSC} < \text{GNN[R]} \equiv \omega\text{-GML}$$

Relativ zu MSO:

$$\text{GNN[F]} \equiv \text{GMSC} \equiv \text{GNN[R]} \equiv \omega\text{-GML} \equiv \text{CMPA}$$

Setting	GNN[F]	GNN[R]
Absolut	$\equiv \text{GMSC (finit)}$	$\equiv \omega\text{-GML (infini)}$
Relativ zu MSO	$\equiv \text{GMSC}$	$\equiv \text{GMSC (!!)}$
Constant-iteration	$\equiv \text{GML}$	$\equiv \text{depth-bounded } \omega\text{-GML}$

Praktische Implikationen

- ✓ **Modellwahl:** Wenn dein Problem in MSO ausdrückbar ist (fast immer der Fall), reicht GNN[F]. Du brauchst kein theoretisch stärkeres Modell.
- ✓ **Theoretische Analysen:** Forscher arbeiten oft mit reellen Zahlen weil es mathematisch einfacher ist. Für MSO-Eigenschaften ist das kein Problem — die Ergebnisse gelten auch für Floats.
- ✓ **Debugging:** Wenn ein GNN eine MSO-definierbare Eigenschaft nicht lernt, liegt es nicht am Modell (Float vs Real) sondern am Training oder der Architektur.

Offene Fragen

Das Paper lässt bewusst einige Fragen offen:

- **Terminierung:** Wie lernt ein RGNN im Training wann er terminieren soll? Das ist noch offen und praktisch sehr wichtig.
- **Globale Readouts:** Appendix D zeigt dass die Charakterisierung auf GNNs mit globalem Readout erweiterbar ist (GMSC + counting global modality).

Vortrag Checkliste

30 Minuten Vortrag — Zeitplan:

- | | |
|--|--------------|
| ☐ Einstieg mit provokanter Frage über Training-Debugging | 3 min |
| ☐ GNN kurz wiederholen (Publikum kennt es) | 2 min |
| ☐ RGNN: ein klares Bild, ein Satz Unterschied | 3 min |
| ☐ Float-Problem: Zahlenbeispiel konkret rechnen ($1 + (-1) + 0.01$) | 5 min |
| ☐ GMSC und ω -GML: nur Intuition, keine formalen Beweise | 5 min |
| ☐ Theorem 3.2 und 3.4 als Ergebnis präsentieren | 3 min |
| ☐ MSO erklären: was kann es ausdrücken? | 3 min |
| ☐ Theorem 4.3: der Twist, Floats reichen für alles Praktische | 4 min |
| ☐ Fazit und Implikationen | 2 min |

Key Takeaways

GNN[F] $\leftrightarrow$ GMSC

Floats entsprechen einer finiten Regellogik mit Zählmodalitäten.

GNN[R] $\leftrightarrow$ ω -GML

Reelle Zahlen entsprechen unendlicher Disjunktionslogik (sehr mächtig, sogar unentscheidbar).

Relativ zu MSO: alles gleich

Die Extramacht liegt wo man sie in der Praxis nie braucht.

Praktische Implikation

Theoretische Analysen mit $\mathbb{R}$ sind trotzdem praxisrelevant — für MSO-Eigenschaften gilt alles auch für Floats.